2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom)

NetANNS: A High-Performance Distributed Search Framework Based On In-Network Computing

Penghao Zhang*†, Heng Pan*‡, Zhenyu Li*‡, Gaogang Xie§ and Penglai Cui*†

*Institute of Computing Technology, Chinese Academy of Sciences, China
†University of Chinese Academy of Sciences, China
‡Purple Mountain Laboratories, China
§Computer Network Information Center, Chinese Academy of Sciences, China
Email: zhangpenghao, panheng, zyli@ict.ac.cn, xie@cnic.cn, cuipenglai20b@ict.ac.cn

***Abstract*—Approximate nearest neighbor search (ANNS) is the most basic and important algorithm in Database, Machine Learning and other applications. With the expansion of cloud computing, the academia focuses on the study of how to optimize distributed frameworks based on approximate nearest neighbor search such as MapReduce, and Memcached. We implement a new distributed ANNS search framework (NetANNS). The main contributions of NetANNS are to accelerate the data preprocessing with programmable switch, and integrate a variety of efficient ANNS algorithms so that it can choose the most suitable algorithm for each datasets. The experiments show that the search efficiency of NetANNS is about 2x than the common distributed ANNS frameworks which are implemented based on the framework of MapReduce.**

***Keywords*-approximate nearest neighbor search, distributed search, in-network computation**

I. Introduction

Nearest neighbor search(NNS) finds the object with the smallest distance to queried object in the dataset. It is generally believed that in high-dimensional Euclidean space it is very expensive to find an accurate nearest neighbor object due to the "death curse" of [22]. Experiments of [30] show that when objects have a high dimensionality (such as greater than 20 dimensions), the performance of the precise method seldom exceeds the forced linear sweep method. The study then fouces on returning objects that are close enough to query object, called approximate nearest neighbor search (ANNS). We also easily get the top-k version for NSS by finding k nearest objects. With the popularization of cloud computing, ANNS in high-dimensional datasets under a distributed framework has been a hot research topic in recent years [11], [15], [16], [31]. The ANNS processing data under the distributed framework would inevitably become complicated with complicated network topo. The existing methods solve it by preprocessing the data or applying more efficient algorithms. But the shortcomings of these methods are low-quality search results and unfair load balance [14], [18].

The paper focuses on using programmable switches to accelerate the ANNS algorithms in a general distributed framework(NetANNS). Inspired by local sensitive hashing(LSH) algorithm [11], [15], [28], we find that programmable switches can praser, extract and classify the data which are carried on packets. We can use the programmable switch which is efficient to classify data through match and action operations, and send data with similar characteristics to the same server for the following search. At the same time, the programmable switch can efficiently process packets and reduce the delay of network communication. We have implemented a general distributed search framework on search servers and integrated a variety of efficient ANNS algorithms. After the search servers receive similar data, they select the most appropriate ANNS algorithm to handle the data. The rest of the paper is structured as follows. Section II presents the background and motivation, Section III describes the design of NetANNS. We evaluate NetANNS in Section IV. Section V surveys related work, and we conclude the paper in Section VI.

II. Background

ANNS problems on high-dimensional data have been extensively studied in Databases, Computer Theories, Computer Vision, Machine Learning and other literature. Now people propose a variety of algorithms to slove the problem from different perspectives. Due to its importance and great challenges, the research in this field is still popular. ANNS under a distributed framework also has been a hot research topic in recent years with the application of cloud computing. In this section, we first briefly introduce the following three types of ANNS algorithms: LSH-based partition algorithms, tree-based space partition algorithms and neighbor-based partition algorithms. Then we introduce the distributed search framework based ANNS algorithms. We finally sketch some of the principles that underlie programmable switch.

A. ANNS Algorithms

LSH-based partition algorithms are typically based on the idea of locality-sensitive hashing (LSH) [15], which maps

978-0-7381-2646-3/21/$31.00 ©2021 IEEE
DOI 10.1109/ISPA-BDCloud-SocialCom-SustainCom52081.2021.00047

a high dimensional point to a low dimensional point via a set of appropriately chosen random projection functions. LSH is widely recognized as one of the most effective methods to index similar items in high dimensional space. A point is mapped to a hash value (e.g. bucket) based on a random projection and discritize method. A set of hash functions, such as inverted list-based scheme [21], trenary-based hash scheme [28], or tree-based scheme [29] are used to ensure the performance of LSH-based algorithms. We would replicate two most recent LSH-based methods with theoretical guarantees: TLSH algorithm [28] and QALSH algorithm [21].

Tree-based space partition has been widely used on the problem of exact and approximate NNS. Generally, the space is partitioned in a hierarchically manner and mainly use compact partitioning schemes. Compact partitioning methods divide the data points into clusters. We also replicate two representative compact partitioning methods: FLANN [26] and Annoy [6]. They are used in a commercial recommendation system and widely used in the Machine Learning and Computer Vision communities.

In general, the neighborhood-based methods build the index by retaining the neighborhood information for each individual data point towards other data points or a set of pivot points. Then various greedy heuristics are proposed to navigate the proximity graph for the given query. Finally, we replicate representative neighborhood based method namely KGraph [13].

B. Distributed Search Framework

With the rapid growth of network data, it has become necessary to optimize ANNS to achieve high throughput and low response time. Most ANNS-based distributed search systems employ two main instance-distributed frameworks: MapReduce and Active distributed hash tables (Active DHT). Both frameworks have data in the form of (key, value) pairs. Then we would describe the framework of MapReduce.

MapReduce [12] is a simple computing model, which is used for batch distributed processing of large-scale dataset and uses a large number of commodity machines. It provides a simple computational abstraction that is divided into three phases. First, the mapping phase reads a set of (Key, Value) pairs from the input source and issues zero or more (Key, Value) pairs associated with each input element separately or in parallel calling the user-defined Mapper function on each input element. Second, the Shuffle phase then pairs all the mapper (Key, Value) pairs that share the same Key and outputs on different groups to the next phase. Finally, in the Reduce phase, a user-defined Reducer function is called for each different group, and zero or more values are emitted independently or in parallel to be associated with the group keys. The emitted (key, value) pairs can then be written to disk or used as input to the mapping phase in subsequent iterations. Hadoop [5], an open source system, is a widely used implementation of this computing framework.

C. In-Network Computation

We want to reduce cost of preprocess on Initial Server and network communication. We turn our efforts to the network itself and hope that switch could process data classification. Recently, many network switches have become capable of providing computational capacity. So we can to implement many functions on programmable switches(such as P4 switch). The chip of the P4 switch has two pipelines and a customizable match-action forwarding engine. With the provided programming language and interfaces, programmers are able to dynamically configure the switch to perform diverse control functions. We can use two multi-stage pipelines, ingress pipeline and egress pipeline to achieve high speed on processing packets. Each pipeline stage has a fixed amount of time to process packets in memory (TCAM and SRAM). We propose NetANNS, a distributed search framework based on in-network computing approach that exploits the computational capacity of programmable switches to accelerate process of data and query. NetANNS can reduce the cost of network communication with programmable switches. NetANNS offloads data and query classifying onto the programmable switches.

III. Implementation

In this section, we describe the details of NetANNS. NetANNS mainly aims to reduce time of preprocessing data and query on distributed system. We first describe the design of NetANNS(include data processing and query processing)(Section III-A). We then introduce the implementation of NetANNS in three parts: Data and Query Preprocessing(Section III-B), Data and Query Classification(Section III-C), and Intelligent Selection Mechanism(Section III-D).

A. Overview of NetANNS

Overall, NetANNS follows the architecture of MapReduce and integrates several ANNS algorithms. NetANNS consists of functional components: Data Processing and Query Processing. Logically, NetANNS connects one Initial Server(IS) and many Search Servers(SS) with the utilization of one programmable switch, and connects many Search Servers and one Agent Server(AS) with one normal switch.

Figure 1 shows how to process data. NetANNS selects TLSH algorithm [28] to preprocess data for extracting features on Initial Server, so that NetANNS can obtain a ternary string that consists of 0, 1 and *. The ternary string is suitable for TCAM matching on programmable switch. NetANNS then sends data and ternary strings of data to the programmable switch. The programmable switch matches ternary strings according to rule tables and contructs packet to carry the matched ternary strings and data to Search

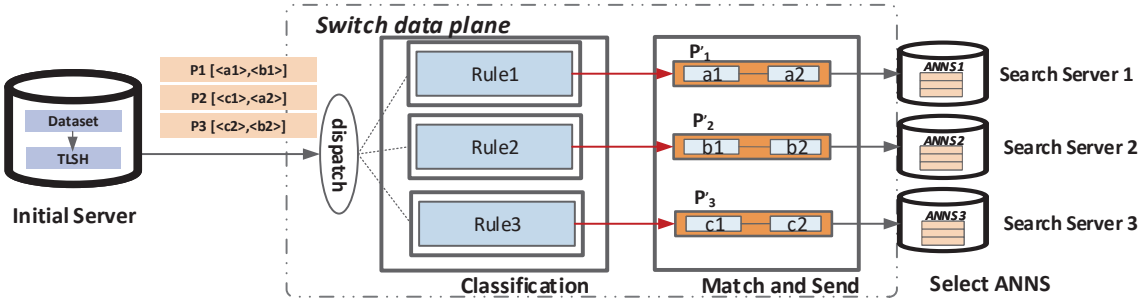

Figure 1. The Overview Of Data Process.

Server. Search Server would select one ANNS algorithm to process data with Intelligent Selection Mechanism. The Query Processing is similar to Data Processing, except for programmable switch match query in different rule tables and send query to different Search Servers to find the answers. When Search Server receive the query, it applies the ANNS algorithm to search top-K answers and sends answers to Agent Server. Agent Server aggregates answers from different Search Servers and sends top-K answers to users.

B. Data and Query Preprocessing

To support data and query classification on programmable switches, NetANNS performs a extension of IP network protocol so that switch can efficiently match and classify data(and query) which are carried on packets. For simplicity, we refer to this as NetANNS protocol. We use two reserved bits in the IP Type of Service(ToS) to indentify NetANNS packets(set ToS 00 as normal packet, 01 as NetANNS data packet, 10 as NetANNS query packet, and 11 as NetANNS control packet). Note that programmable switches intercept and process NetANNS protocol packets based on the value of ToS, and other packets would be forwarded directly because their ToS is 00. Therefore, NetANNS would not affect normal network functions.

To restate, the primary mission of the Initial Server is to preprocess data with some functions(such as LSH algorithm or hash) and classify data into Search Servers randomly or regularly. When the scale of the distributed search system increases, the Initial Server sends more data. The procedure is the bottleneck of distributed search framework. NetANNS mainly uses programmable switches to classify data so that reduce the process time of Initial Server. And on programmble switches, NetANNS can autonomously configure the rule tables which are used to classify. We need to convert data in advance into one format which can be identified and matched by the programmable switch. The programmable switches currently support ternary match, exact match, and range match. Therefore, we select TLSH algorithm which is commonly used in hardware to convert data into ternary strings and uses strings to match query and find top-K answers.

NetANNS preprocesses the data by TLSH on Initial Server, carries out them into a NetANNS data packet and sends packet to the switch. Due to quick classification of programmable switches, NetANNS can greatly improve the performance of preprocessing data which is only processed on Initial Server. After classifying, data with similar features are sent to same search server, so NetANNS can use suitable ANNS algorithm to store them for searching. In addition, the queries can be handled in the same way, and NetANNS would use similar ternary matching to match features of queries on the switch. When 70% up of features of one query are matched, NetANNS would send the query to the corresponding Search Server. The similar ternary matching not only guarantees the correct rate of search, but also reduces some unnecessary search.

C. Data and Query Classification

When Initial Server send packet into programmable switch, switch parser pakcet and extract ToS field of IP header of packet to judge if packet belong to NetANNS format. If ToS is 01, 10 or 11, packet would be processed by switch. Otherwise switch would forward packet directly. First, after receiving the packet, the switch parses the header of the packet and judges if the packet is NetANNS packet or not according to the ToS. If it is a normal packet, the switch would forward it normally.

If ToS is 01, the packet belongs to NetANNS data packet. Switch extracts TLSH strings of data, and then performs ternary match with rule tables. If the TLSH string are hit on one entry of rule tables, switch would construct a NetANNS data packet to carry the data and TLSH string, then send them into the corresponding Search Server according of route table stored on switch. The switch would extract TLSH string and data in sequence, and match the string with rule tables according to its key and mask. If string is matched, switch would send corrosponding data into Search Server.

If ToS is 10, the packet is NetANNS query packet. Switch also extract TLSH string of the query and performs similar matching with rule tables. If the degree of similarity is greater than or equal to 70%, switch thinks the query is hit the entry of rule tables and construct packet to carry out query to the corresponding Search Server. Note that the

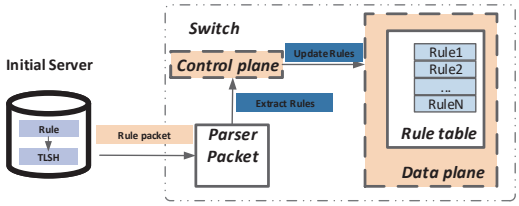

Figure 2. Updating Rule Tables On Switch

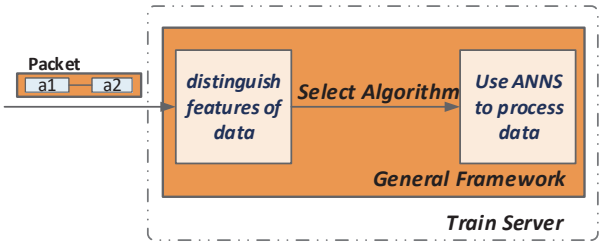

Figure 3. Intelligent Selection Mechanism

switch does not support similar ternary matching. NetANNS would disable the 30% bits(which are set 1) of mask of TLSH string to get many different queries. Then switch process these queries in sequence to perform ternary match. If one entry is hit and the query is sent to corresponding Search server, the later queries could not be send the Search server even they are hit the same query. So that NetANNS avoids Repeated searching.

Finally if it is a NetANNS control packet(the ToS is set 11), switch would handle it with the control plane. Figure 2 shows the process of how to parse control packet and update rule tables on switch. For different datasets we need to consider switch to use different rule tables to classify them. Therefore, NetANNS could send control packets which are constructed by initail server to switch for updating rule. in the data preprocessing with TLSH on the IS, we would mark the data, send the NetANNS control package to the switch in advance and update appropriate rulesets on switch. The switch would extract rules of packet and update them into rule tables by using control plane. The control plane applies updating functions with python code. For typical datasets such as image dataset, text dataset and audio dataset, Initial Server would process them and get rules offline for fast updating. The rules are also TLSH strings which distinguish dataset and cluster data which have same features. For example, rule A is 11***, and A can cluster data where TLSH strings are subset of 11***.

D. Intelligent Selection Mechanism

Based on the previous description, NetANNS sends data with similar features to the same Search server. NetANNS implements an Intelligent Selection Mechanism(ISM) to select the appropriate algorithm from framework to process received data. NetANNS implements a general framework, and an Intelligent Selection Mechanism(ISM) to select the appropriate algorithm from framework to process received data. Figure 3 shows data flow of the ISM and framework.

We first introduce the search framework of NetANNS. NetANNS implements a universal input function and output function to convert data into the form that can be recognized by corresponding ANNS algorithms. Then NetANNS has implemented ANNS algorithms which are introduced on Section II-A. So NetANNS would select algorithm A according to ISM. Then NetANNS converts the received data into the format required by ANNS algorithm A through input interface of framework. And NetANNS would use algorithm A to process the data and construct the datastructure. When queries are sent to Search server, NetANNS also processes query with algorithm A. Then NetANNS converts the answers that are searched by algorithm A into the uniform format(consist of the ID of data vector, the value which indicates distance of query and data vector)through the output function. Then we send answers to the Agent Server. If NetANNS wants to add a new ANNS algorithm on framework, it only need to add the corresponding data conversion at the input function and answers conversion at the output function. Then NetANNS compile new algorithm and transplant the algorithm into framework.

Then we introduce the intelligent selection mechanism (ISM). ISM is mainly used to select the appropriate ANNS algorithm for each Search Server which may receive data with different features. We know that the hash-based ANNS algorithm is suitable for processing sparse or normal dataset, and the tree-based or graph-based ANNS algorithm are suitable for processing dense dataset. Therefore, when Search Server receives data, NetANNS would extract TLSH strings and data on Search Server to ISM. ISM would determine the type of data accord to the feature distribution, and select the appropriate ANNS algorithm. First, we calculate the distribution of features in advance which indicates the number of character 0 in each bit of the TLSH string. Then ISM selects a suitable algorithm according to the proportion of the number of 0 in each bit of the distribution, and applies the corresponding algorithm to process received data. If the the proportion of the number of 0 is 0-30%, ISM would apply FLANN algorithm.(applying KGraph for 30-45%, applying Annoy for 45-60%, applying TLSH for 60-70%, applying QALSH for 75-100%). And ISM supports to change the way of algorithm selection, and add a new selected way when a new algorithm is transplanted.

IV. Experimental Evaluation

In this section, we will evaluate NetANNS. As a baseline, we compare it with two alternatives: Layered-LSH [7], a traditional MapReduce-based variant of the LSH algorithm; and a distributed framework for integrated multiple ANNS algorithms implemented under MapReduce [12], we call it MapANNS.

A. Evaluation Methodology

Testbed Setup. We evaluate NetANNS on a testbed with 6 servers, each of which is equipped with 32 cores of Intel(R) Xeon(R) E5-2682 CPU @ 2.5GHz, 256GB RAM with Ubuntu 16.04 and Linux kernel 4.15.0-132. All servers are directly connected to a Barefoot Tofino switch (3.2 Tb/s). Additionally, we run 4 virtual machines (VMs) on each physical server where each VM monopolizes 4 CPU cores and occupies 32GB memory.

Workload and Datasets. The experiments involve four different types of datasets: a Random dataset, two Text datasets, one Image dataset and an Audio dataset, as described below:

1) **Random** Random dataset is constructed by sampling points from the normal distribution $N^d(0,1)$ with $d = 100$. The dataset contains 1000M data points. The queries are generated by adding a small perturbation to the positive distribution $N^d(0,r)$ with $r = 0.3$. In total, we have 10M queries. This type of dataset has been used in many ANNS algorithm experiments [7], [28]. Thus we also use it to solve the (c,r)-NN problem with $c = 2$. The expected distance for each query point to its closest data point is $r = 0.5$, and high-probability data point is also within the distance $c * r$
2) **Text** We use two real-world text datasets: Enron [3] and Glove [4]. Enron and Glove, where both of them contains about 1000M data points. Enron origins from a collection of emails while each Glove data item consists of 100-dimensional word feature vectors extracted from Tweets. For the three text datasets above, we respectively generate 10M queries with $d = 100$.
3) **Image** We adopt one image dataset. It is a Deep [2] dataset, which contains deep neural codes of natural images obtained from the activations of a conventional neural network and generates 1000M 256-dimensional data points with 10M queries.
4) **Audio** In Audio [1] dataset, it mainly consists of 192-dimensional audio feature vectors extracted by Marsyas library from DARPA TIMIT audio speed dataset, which generates 1000M data points and 10M queries.

We are particularly interested in four aspects: (i) the performance improvement of NetANNS comparing with the baselines(Section IV-B); (ii) the effect of each major component (i.e. Classificatin and Selection Mechanism) (Section IV-C); (iii) the impact of updating rule tables (Section IV-D); (iv) the search quality of NetANNS (Section IV-E);

B. Evaluating NetANNS System

As mentioned before, NetANNS mainly uses programmable switches to accelerate data(and query) processing. Then, we evaluate NetANNS and compared it with LayerLSH and MapANNS solutions using the four types of datasets. The setup is the same as that in Section IV-A. Figure 4 and Figure 5 shows the experimental results. Indeed, NetANNS can achieve a significant performance improvement comparing with the two baselines under the all types of datasets. This is because the programmable switch extracts TLSH strings of data, matches and classifies them into different Search Servers, and then NetANNS use ISM to select best algorithm to process data(and query) on Search Servers. The improvement stems from four aspects. The first is programmable switch has better performance than software on classifying data. The second is that the processing of the Search Server because Search Server receives data with similar features, NetANNS is easily to find answers when process query and avoid unnecessary search. The third reason is that ISM would select appropriate algorithm to apply on NetANNS according the infomation of data.

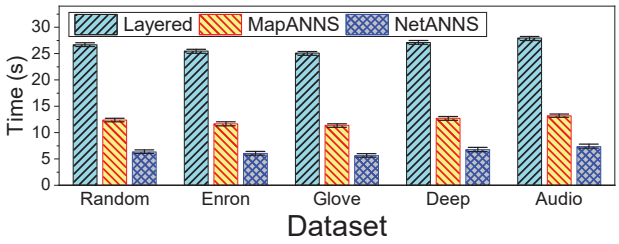

Figure 4. The Total Time On Processing Data.

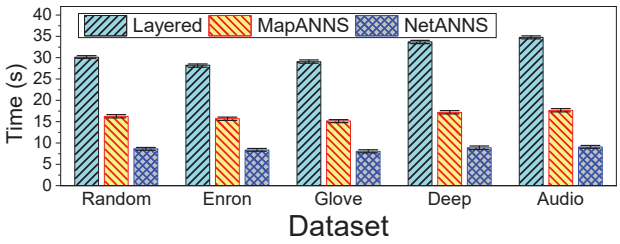

Figure 5. The Total Time On Processing Query.

Therefore, the optimization of switch would not only have benefits for network but also end host.To better highlight for the processing improvements on NetANNS. We report the data processing time in Figure 4 and 10M queries processing time in Figure 5. We see that both of processing time are reduced by about 4-5x comparing with Layerd-LSH and 1.8-2.3x comparing with MapANNS.

C. Breakdown Analysis NetANNS.

Indeed, NetANNS can achieve better search performance compared with the baselines(e.g. Layered-LSH and MapANNS). In the section of experiments, we discuss the role played by NetANNS's two components: Data(and Query) classification and Intelligent Selection Mechanism. In this set of experiments, we respectively use $Baseline$ to refer to the solution where both components are disabled(i.e. the switch only forward packets and Search Server only

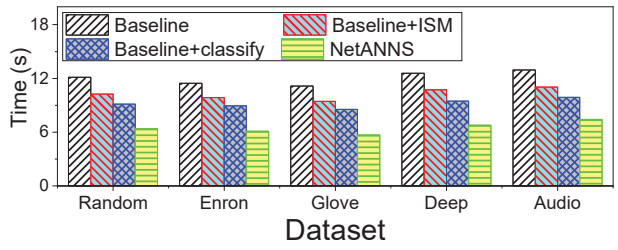

Figure 6. The Performance Improvement On Optimizing Data Process.

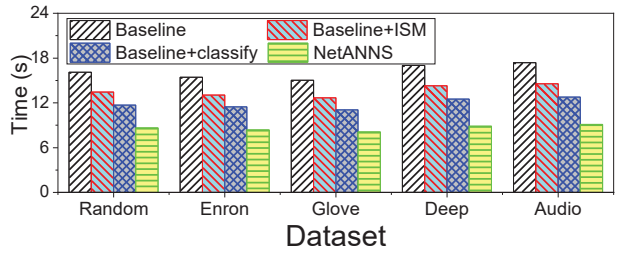

Figure 7. The Performance Improvement On Optimizing Query Process.

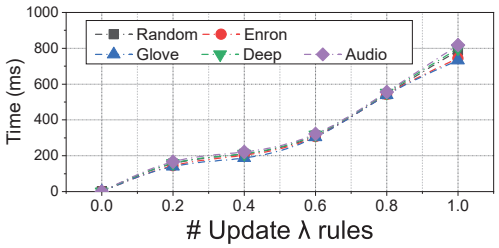

Figure 8. The Latency of Updating Rule Tables On Switch.

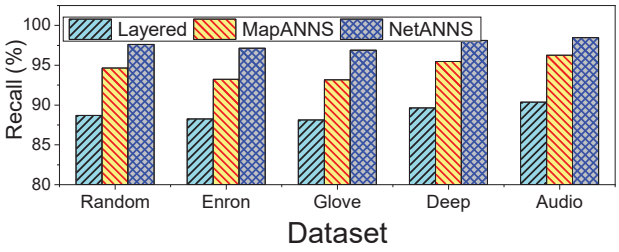

Figure 9. The Recall Rates on Different Search Frameworks

apply LSH algorithms), $Baseline + classify$ to refer to the solution where the programmable switch also performs component of classification, and $Baseline + ISM$ to refer to the solution where NetANNS performs ISM. Finally, NetANNS uses both components.

Figure 6 and Figure 7 shows data and query process time of NetANNS packets when it respectively adopts the four solutions. We find that the classification can reduce the process time of both to about 60% of the Baseline, while ISM can reduce to about 40% of the Baseline. We see they both can save the processing time greatly. Note that the time reduction by the classification is due to the performance of switches, while the time reduction by ISM comes from that NetANNS could quickly process similar data and reduce unnecessary search.

D. Impact Of Updating Rule Tables Of NetANNS

The performance of NetANNS is naturally impacted by the rule tables. Hence, we next discuss the impact of updating time on tables. NetANNS need to use different tables to classify different datasets on switches. Although the update frequency of NetANNS is not high, it requires time of the updating tables is as small as possible. NetANNS can also use the same tables for classification if the cost of updating is large, but it would reduce efficiency of searching for different datasets.

Figure 8 plots the results. We can see that updating tables increase a little delay of NetANNS. That is because NetANNS sends control packet with rules and uses the control plane of the switch to update tables. This operation is similar to process data packet on switch, and only takes up the delay of processing several packets. NetANNS need updating tables when it processes new dataset, so the update frequency is very low and the operation would not affect the overall performance improvement.

E. Search Quality

The above experiments show that NetANNS is able to significantly accelerate distributed search. Next we need to verify that the search quality is not harmed but a little better than other two solutions. The experimental setting is the same as those of the previous experiments. Figure 9 and Figure 10 compares the search quality of three solutions. The search quality is measured by the recall rates and F1-scores. For both the recall rates and the F1-scores, NetANNS achieves better search quality (exceeding 90%) with other two typical distributed search implementation. This is because NetANNS does not break any ANNS algorithm and use ISM to select appropriate ANNS algorithm on each Search Server to process data and query. Thus the search quality of NetANNS could get better quality than other framework with fixed ANNS algorithms.

V. Related Work

Approximate Nearest Neighbor Search. In the last decade, the nearest neighbor search (NNS) problem has gained widespread attention and become the basis for many applications, such as databases, computer vision, multimedia, machine learning, and recommendation systems. However, NNS algorithm does not perform well in high-dimensional data [10], [27]. Therefore, approximate nearest neighbor (ANNS) search has been widely used [19], [21], [25], and its methods can be divided into three aspects. First is spatial partitioning, including all approaches based on trees, such as

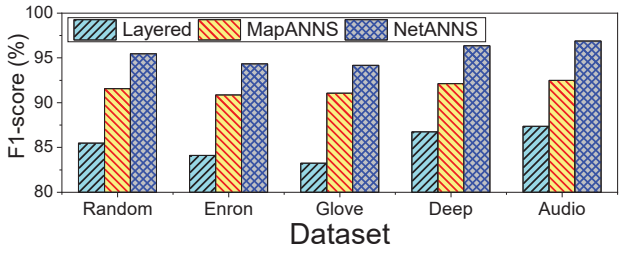

Figure 10. The F1_scores on Different Search Frameworks.

R-tree [17] and K-D tree [9]. FLANN [26] selects the most appropriate algorithm for the specific data set. Annoy [6] is an empirically engineered algorithm that has been used in the recommendation engine, and has been recognized as one of the best ANN libraries. Second is a hashed method that returns an approximate nearest neighbor to the query point in exchange for efficiency. QALSH [21] introduced the concept division of query-aware bucket and developed a new query-aware LSH function. Ternary Locality Sensitive Hashing (TLSH) [28] also is a variant of LSH, The main idea is to construct TLSH functions that can project any d-point $p \in R^d$ into the set 0,1,*. Third is domain-based methods. KGraph [13] is a representative technique for KNN graph construction and approximate nearest neighbor search.

Distributed Search System. For large applications, ANNS needs to handle large data. This means that the single ANNS algorithm which runs on server described above does not work well in all aspects such as accuracy, recall rate, and efficiency. Bawa [8] proposed a data structure named LSH Forest. Hua [20] focus on distributed file systems with local and non-uniform distribution.

Based on the above work, this paper proposes NetANNS, which uses programmable switch to optimize the preprocessing data process and reduce the network overhead. This is because we observe that networks constitute a key bottleneck in distributed search [24]. Related methods have been proposed to balance the load between different servers [23] or to reduce the number of network calls [7] to reduce the network burden. However, the efficiency of these optimization methods is mainly concentrated in the terminal system (server side). Due to the increasing data size and concurrent query volume, the network is still the performance bottleneck of distributed search system. To address this challenge, NetANNS proposed a new approach that utilizes in-network computing to reduce communication overhead (on the network side) and reduce workload on the end system.

VI. Conclusion and Furture Work

This paper presents a new method of NetANNS, which uses programmable switch to accelerate data and query processing on distributed search system. As a key design requirement, NetANNS has not changed the architecture of the distributed search system, but optimized it in three aspects: (1) NetANNS offloads the classification process of data(and query) preprocessing into the programmable switch and uses switch to classify data into many Search Servers. (2) NetANNS designs a framework which integrates many ANNS algorithms. (3) NetANNS would select the optimal ANNS algorithm for different datasets to improve efficiency with ISM.

NetANNS uses programmable switch and ANNS framework to accelerate the distributed search system. We also work on further acceleration of distributed search by offload NetANNS into Smart NICs to improve efficiency of classifying data. Finally, we also are interesting to evaluate NetANNS in large distributed search system with large data.

Acknowledgments

We thank the anonymous reviewers for their insightful feedback. This work is supported in part by National Key R&D Program of China (Grant No. 2019YFB1802800) and the Natural Science Foundation of China (Grant No. 62002344).

References

[1] Audio. http://www.cs.princeton.edu/cass/audio.tar.gz.

[2] Deep. https://yadi.sk/d/I_yaFVqchJmoc.

[3] Enron. https://www.cs.cmu.edu/~./enron/.

[4] Glove. https://nlp.stanford.edu/projects/glove/.

[5] Hadoop. http://hadoop.apache.org, 2007.

[6] Annoy. Annoy at github https://github.com/spotify/annoy, 2015.

[7] Bahman Bahmani, Ashish Goel, and Rajendra Shinde. Efficient distributed locality sensitive hashing. In *Proceedings of the 21st ACM international conference on Information and knowledge management*, pages 2174–2178. ACM, 2012.

[8] Mayank Bawa, Tyson Condie, and Prasanna Ganesan. Lsh forest: Self-tuning indexes for similarity search. In *Proceedings of the 14th International Conference on World Wide Web*, WWW '05, page 651660, 2005.

[9] Jon Louis Bentley. K-d trees for semidynamic point sets. In *the sixth annual symposium*, 1990.

[10] Christian, B246, hm, Stefan, Berchtold, Daniel, A., and Keim. Searching in high-dimensional spaces: Index structures for improving the performance of multimedia databases. *Acm Computing Surveys*, 2001.

[11] M. Datar. Locality-sensitive hashing scheme based on p-stable distributions. In *Proc. of the 20th ACM Symposium on Computational Geometry, 2004*, 2004.

[12] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):p.107–113, 2008.

[13] Wei Dong, Moses Charikar, and Kai Li. Efficient k-nearest neighbor graph construction for generic similarity measures. In *International Conference on World Wide Web*, 2011.

[14] Christos Doulkeridis, Akrivi Vlachou, Yannis Kotidis, and Michalis Vazirgiannis. Peer-to-peer similarity search in metric spaces. In *International Conference on Very Large Data Bases*, 2007.

[15] A. Gionis. Similarity search in high dimensions via hashing. In *Proc of Vldb Conference*, 1999.

[16] Y. Guo, Q. Zhuge, J. Hu, M. Qiu, and H. M. Sha. Optimal data allocation for scratch-pad memory on embedded multi-core systems. In *International Conference on Parallel Processing*, 2011.

[17] A. Guttman. R-trees: a dynamic index structure for spatial searching. *ACM SIGMOD Record*, 1984.

[18] Y. Hua and K. Gai. An empirical study on preprocessing high-dimensional class-imbalanced data for classification. In *IEEE International Conference on High Performance Computing & Communications*, 2015.

[19] Y. Hua, K. Gai, and Z. Wang. A classification algorithm based on ensemble feature selections for imbalanced-class dataset. In *IEEE International Conference on Big Data Security on Cloud*, 2016.

[20] Yu Hua, Bin Xiao, Dan Feng, and Bo Yu. Bounded lsh for similarity search in peer-to-peer file systems. In *International Conference on Parallel Processing*, 2008.

[21] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *Proceedings of the VLDB Endowment*, 9(1):1–12, 2015.

[22] P. Indyk. Approximate nearest neighbors : Towards removing the curse of dimensionality. In *Proceedings of the 30th ACM Symposium on Theory of Computing (STOC'98), May*, 1998.

[23] Yubin Kim, Jamie Callan, J Shane Culpepper, and Alistair Moffat. Load-balancing in distributed selective search. In *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*, pages 905–908. ACM, 2016.

[24] Hangyu Li, Sarana Nutanong, Hong Xu, Foryu Ha, et al. C2net: A network-efficient approach to collision counting lsh similarity join. *IEEE Transactions on Knowledge and Data Engineering*, 31(3):423–436, 2018.

[25] Jinfeng Li, Xiao Yan, Jian Zhang, An Xu, James Cheng, Jie Liu, Kelvin KW Ng, and Ti-chung Cheng. A general and efficient querying method for learning to hash. In *Proceedings of the 2018 International Conference on Management of Data*, pages 1333–1347. ACM, 2018.

[26] M Muja and D. G Lowe. Scalable nearest neighbor algorithms for high dimensional data. *Pattern Analysis Machine Intelligence IEEE Transactions on*, 36(11):2227–2240, 2014.

[27] M. Qiu, Z. Chen, and M. Liu. Low-power low-latency data allocation for hybrid scratch-pad memory. *IEEE embedded systems letters*, 6(4):69–72, 2014.

[28] Rajendra Shinde, Ashish Goel, Pankaj Gupta, and Debojyoti Dutta. Similarity search and locality sensitive hashing using ternary content addressable memories. In *Acm Sigmod International Conference on Management of Data*, 2010.

[29] Yifang Sun, Wei Wang, Jianbin Qin, Ying Zhang, and Xuemin Lin. Srs: Solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index. In *Proceedings of the VLDB Endowment*, 2014.

[30] Roger Weber, Hans-Jorg Schek, and Stephen Blott. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In *VLDB'98, Proceedings of 24rd International Conference on Very Large Data Bases, August 24-27, 1998, New York City, New York, USA*, 1998.

[31] Cui Yu, Beng Ooi, Lee Tan, and H. Jagadish. Indexing the distance: An efficient method to knn processing. 07 2001.